

TP N°1**Premiers pas avec Android**
(Kotlin & ViewBinding)**Matière : Programmation Mobile****Niveau : ING1 Info, L3 Info, L3 Tic****AU : 2026-2027**

- Objectifs :**
- Installer et configurer **Android Studio** avec **Kotlin**.
 - Créer et lancer un premier projet Android.
 - Configurer un Android Virtual Device (**AVD**).
 - Manipuler les **ressources** (textes, couleurs, styles).
 - Créer une interface simple (formulaire de connexion) avec **EditText** et **Button**.
 - Rendre l'application **interactive** avec des clics et des **Toasts**.
 - Explorer des fonctionnalités complémentaires comme l'ajout d'un deuxième écran, la traduction de l'interface et la personnalisation du thème.

Partie 1 : Création et exploration d'un premier projet Android

1. Ouvrez **Android Studio** et Créez un nouveau projet Android :
New Project → **Empty Views Activity**.
Ou
File → **New** → **New Project** → Choisissez **Empty Views Activity**.
 - Renseignez les informations du projet :
 - Nom du projet** : TP1_votreNometPrénom
 - Package name** : isimm.ing1X.tp1_nomprenom (*X est votre groupe*)
 - Langage** : Kotlin
 - Minimum SDK** : API 26 (Android 8.0)
2. Cliquez sur Finish. Le projet créé comporte déjà une activité nommée MainActivity.

NB : Pour tester une application Android, vous pouvez utiliser un émulateur configuré via Device Manager ou un smartphone Android avec le débogage USB activé. Référez-vous aux indications données dans le TP0 pour la configuration.

Suite à la création d'un nouveau projet Android, une fenêtre à gauche est ouverte affichant l'arborescence du projet.

3. Explorez l'arborescence du projet et identifiez :
 - AndroidManifest.xml : configuration globale de l'application.
 - Les classes d'activités, notamment MainActivity.kt.
 - Le code source Kotlin sous java/.
 - Les interfaces XML sous res/layout/, notamment activity_main.xml.
 - Les ressources sous res/values/, notamment strings.xml et colors.xml.

Partie 2 : Découverte et Définition des ressources Android

Pour garantir une bonne architecture et afin de faciliter la maintenance et la réutilisation., toutes les chaînes de caractères, couleurs et dimensions doivent être centralisées dans le dossier `res/values/`.

1. Ouvrir `res/values/strings.xml` et ajouter :

```
<resources>
    <string name="app_name"> TP1_votreNometPrénom</string>
    <string name="mytext">Welcome to My First Android Application</string>
    <string name="hint_username">Nom d'utilisateur</string>
    <string name="hint_password">Mot de passe</string>
    <string name="btn_login">Se connecter</string>
    <string name="msg_success">Connexion réussie</string>
    <string name="msg_error">Erreur: veuillez réessayer</string>
</resources>
```

Toutes les chaînes affichées dans les `TextView`, les hints des `EditText` et les `Button` devront être définies dans `strings.xml`.

2. Dans `res/values/colors.xml`, personnaliser :

```
<color name="mycolor">#FF5722</color>
<color name="primaryColor">#6200EE</color>
<color name="buttonColor">#03DAC5</color>
```

3. Créez le fichier `res/values/dimensions.xml` : clic droit sur `values` → `New` → `Values Resource File`, puis saisissez `dimensions` comme nom du fichier.

```
<resources>
<dimen name="mydimen">40sp</dimen>
</resources>
```

Partie 3 : Première manipulation de l'interface et des ressources

1. Lancez l'application en cliquant sur Run . Observez le texte « Hello World ! » affiché sur l'activité.
2. Ajoutez un `TextView` ayant pour id `mytextview` et comme texte initial « This is My Android App ».
3. Modifiez ensuite son texte afin qu'il utilise la ressource `@string/mytext`.

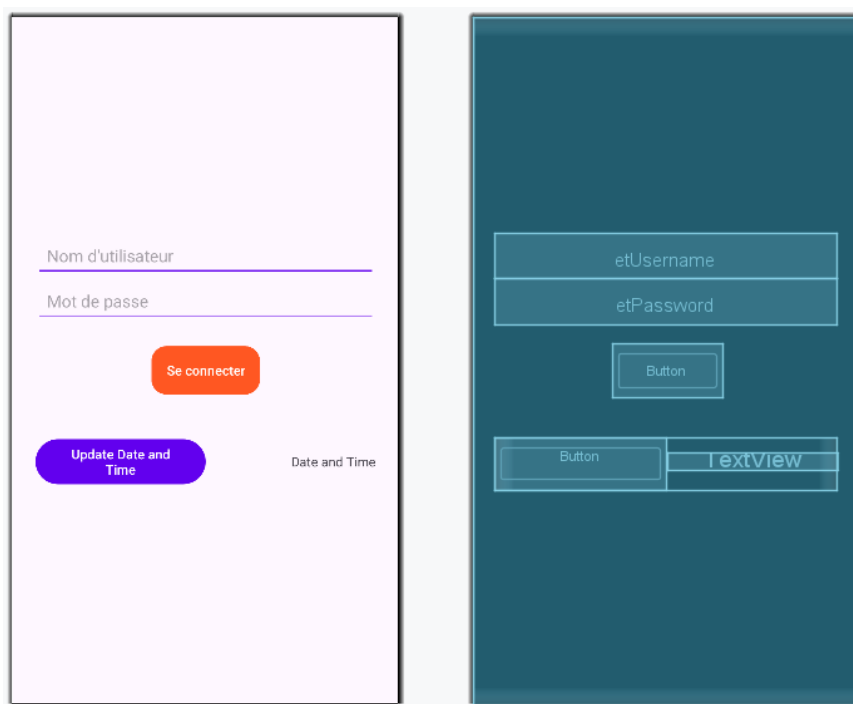
4. Attribuez la couleur bleue (#0000FF) au TextView mytextview et au message « Hello World ! ».
5. Remplacez ensuite cette couleur par la ressource @color/mycolor.
6. Mettez la taille du texte « Welcome... » à 35sp et celle du texte « Hello... » à 50sp.
7. Remplacez ces tailles par la ressource @dimen/mydimen.
8. Dans MainActivity.kt, affichez au lancement un Toast contenant « This app is developed by X », où X correspond à votre nom et prénom. Le Toast doit être placé dans onCreate(), après setContentView().

```
Toast.makeText(context: this, text: "This App is developed by ...", Toast.LENGTH_LONG).show()
```

NB : Le but de cette partie est de comprendre qu'une interface Android peut être construite à partir de vues XML et que leurs propriétés peuvent être alimentées par des ressources.

Partie 4 : Conception de l'interface d'authentification

1. Créer une interface de formulaire d'authentification dans activity_main.xml à l'aide de l'éditeur visuel Design (drag & drop) comme montre la figure suivante :



Étape 1 : Ouvrir l'éditeur de layout

1. Dans le **Project Explorer**, ouvrir : **app** → **res** → **layout** → **activity_main.xml**.
2. Deux vues disponibles : Design (drag & drop) et Code (XML). Cliquer sur l'onglet **Design** en haut de la fenêtre (au lieu de Code).
3. Supprimer le TextView "Hello World" déjà présent

Étape 2 : Choisir un conteneur

1. Dans la Palette (à gauche), chercher Layouts et Faire glisser un LinearLayout (Vertical) sur l'écran.

2. Dans l'onglet Attributes, configurez :

```
width = match_parent
height = match_parent
orientation = vertical
gravity = center
padding = 24dp
```

Étape 3 : Ajouter les champs de saisie

1. Depuis la Palette → Text, glisser un Plain Text (champ texte) dans le layout.
2. Dans Attributes, régler :

```
id = @+id/etUsername
hint = Nom d'utilisateur (ou @string/hint_username si déjà défini).
width = match_parent
padding = 12dp
```

3. Ajouter un deuxième Plain Text (champ texte).

4. Changer inputType → textPassword.

```
id = @+id/etPassword
hint = Mot de passe (ou @string/hint_password si déjà défini).
width = match_parent
padding = 12dp
```

Étape 4 : Ajouter un bouton

1. Depuis la Palette → Buttons, glisser un Button en dessous des champs.
2. Dans Attributes, régler :

```
id = @+id/btnLogin
text = Se connecter (ou @string/btn_login).
backgroundTint = choisir une couleur (@color/buttonColor).
layout_marginTop = 20dp
```

Étape 5 : Tester le rendu

Après avoir construit l'interface en mode Design, ouvrez l'onglet Code et observez le XML généré automatiquement. Modifiez ensuite directement une propriété dans le XML et vérifiez le résultat visuel.

1. En haut de l'éditeur, choisir un modèle de téléphone (ex. Pixel 6).
2. Vérifier que l'interface ressemble au formulaire de connexion (figure ci-dessous).
3. Une fois votre interface réalisée avec le mode Design (drag-and-drop), ouvrez l'onglet Code en bas de l'éditeur (activity_main.xml).

4. Observez le code XML généré automatiquement par Android Studio.
5. Modifiez le texte et la couleur du bouton dans le code XML et observez le résultat.

Étape 6 : Date et heure courantes

Enrichissez l'interface d'authentification en ajoutant :

- Un TextView en bas à droite pour afficher la date et l'heure courantes.
 - Un bouton « Update Date and Time » en bas à gauche.
1. Lors du chargement de l'activité, affichez la date et l'heure courantes.
 2. Lors du clic sur « Update Date and Time », actualisez cette valeur.
 3. Utilisez la classe Date pour réaliser cette fonctionnalité.

NB : La date et l'heure constituent une exception : elles ne sont pas stockées dans strings.xml.

Partie 5 : Logique en Kotlin et gestion des événements

Dans cette partie, vous découvrirez comment rendre votre application interactive en capturant les clics sur les boutons avec `setOnClickListener()` et en affichant des messages contextuels via Toast, pour informer l'utilisateur du résultat de son action.

1. Authentification :

Lorsque l'utilisateur saisit un login et un mot de passe puis clique sur « Se connecter » :

- Si le couple est correct, affichez un Toast indiquant que les données sont correctement saisies.
- Sinon, affichez un Toast indiquant que les informations saisies sont erronées.

Utilisez la règle d'authentification suivante : le mot de passe doit être égal au login précédé de « pw ». Par exemple, si le login est **ali**, le mot de passe correct est **pwali**.

- Pour se faire, ouvrir **MainActivity.kt** et ajouter correctement le code suivant :

```
import android.widget.Button
import android.widget.EditText
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etUsername = findViewById<EditText>(R.id.etUsername)
        val etPassword = findViewById<EditText>(R.id.etPassword)
        val btnLogin = findViewById<Button>(R.id.btnLogin)

        btnLogin.setOnClickListener {
            val username = etUsername.text.toString()
            val password = etPassword.text.toString()
```

```
        if (password == "pw" + username) {  
            Toast.makeText(this, getString(R.string.msg_success),  
                Toast.LENGTH_SHORT).show()  
        } else {  
            Toast.makeText(this, getString(R.string.msg_error),  
                Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

2. Mise à jour de la date et de l'heure

Associez un `setOnClickListener()` au bouton « Update Date and Time » et utilisez `Date` pour mettre à jour le `TextView` correspondant.

```
updateButton.setOnClickListener {  
    dateTextView.text = Date().toString()  
}
```

Partie 6 : Test et exécution

1. Lancer l'application sur l'émulateur ou sur smartphone réel (USB).
2. Vérifiez que le Toast de bienvenue apparaît au lancement.
3. Tester plusieurs couples login / mot de passe :
 - **admin / pwadmin** → affiche **Connexion réussie**
 - **admin / 1234** → affiche **Erreur : veuillez réessayer**
4. Testez le bouton « Update Date and Time » et vérifiez que la valeur affichée est actualisée.
5. Modifiez le texte ou la couleur du bouton “Se connecter” directement dans le fichier **activity_main.xml** ou dans le **code Kotlin**, puis lancez l'application pour observer le changement.
6. Expliquez brièvement comment la modification dans le XML ou le code Kotlin affecte l'affichage.
7. Selon vous, quel est le rôle du lien entre XML ↔ Kotlin ↔ interface visuelle dans une application Android ?

Partie 7 : Utiliser ViewBinding à la place de findViewById

Google recommande aujourd'hui **ViewBinding** comme alternative moderne à `findViewById`, car il évite les erreurs et simplifie le code.

Pour avoir accès aux objets graphiques, il faut :

- a. **Activer ViewBinding** : Dans le fichier **build.gradle (Module: app)**, ajoutez la configuration suivante et n'oubliez pas de cliquer ensuite sur **Sync Now**.

```
android {  
    ...  
    buildFeatures {  
        vie Binding = true  
    }  
}
```

- b. **Modifier votre activité** : Dans **MainActivity.kt**, utilisez le binding généré pour l'interface et remplacez `setContentView` par :

```
private lateinit var binding: ActivityMainBinding  
  
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
}
```

- c. **Accéder ensuite aux vues directement** :

```
binding.etUsername  
binding.etPassword  
binding.btnLogin
```

- Remplacez progressivement les appels `findViewById` par les références binding correspondantes.

Exemple avec un bouton :

```
binding.btnLogin.setOnClickListener {  
    Toast.makeText(this, getString(R.string.msg_success), Toast.LENGTH_SHORT).show()  
}
```

ou

```
binding.btnLogin.setOnClickListener {  
    Toast.makeText(this, "Connexion réussie !", Toast.LENGTH_SHORT).show()  
}
```

⇒ Ainsi, plus besoin d'utiliser findViewById, chaque vue est accessible directement via binding.nomDeLaVue.

Partie 8 : internationalisation

L'objectif dans cette partie est de rendre l'application multilingue : vous allez apprendre à adapter votre application pour plusieurs langues en traduisant les chaînes de caractères, ce qui permettra à l'interface de s'afficher automatiquement dans la langue du téléphone.

1. Créer un fichier de ressources pour l'anglais :
 - Dans **res/**, créer un nouveau dossier : **values-en**.
 - Ajouter un fichier **strings.xml** dans **res/values-en**.
2. Définir les traductions :
 - Copier les chaînes depuis **res/values/strings.xml** et traduire comme ci-dessus :

```
<resources>  
    <string name="app_name">LoginApp</string>  
    <string name="hint_username">Username</string>  
    <string name="hint_password">Password</string>  
    <string name="btn_login">Login</string>  
    <string name="msg_success">Login successful</string>  
    <string name="msg_error">Error: please try again</string>  
</resources>
```

3. Tester la langue
 - Changer la langue du téléphone ou de l'émulateur en anglais.
 - Vérifiez que les textes de l'application sont automatiquement affichés en anglais.

Partie 9 : Personnaliser le thème avec plus de couleurs et styles

1. Ouvrir **res/values/colors.xml** et changer la couleur du bouton en #FF5722 et la couleur primaire et secondaire comme suit :

```
<color name="primaryColor">#6200EE</color>  
<color name="secondaryColor">#03DAC5</color>
```

2. Appliquer les couleurs dans le thème :

- Ouvrir **res/values/themes.xml** et remplacer les références par les nouvelles couleurs :

```
<style name="Theme.LoginApp"
parent="Theme.Material3.DayNight.NoActionBar">
    <item name="colorPrimary">@color/primaryColor</item>
    <item name="colorSecondary">@color/secondaryColor</item>
</style>
```

3. Créer un style personnalisé pour le bouton dans **res/values/themes.xml**

```
<style name="CustomButton" parent="Widget.Material3.Button">
    <item name="cornerRadius">16dp</item>
    <item name="android:textColor">#FFFFFF</item>
    <item name="android:padding">16dp</item>
</style>
```

4. Appliquer le style au bouton dans **activity_main.xml** en ajoutant :

```
style="@style/CustomButton"
```

5. Lancer l'application et vérifier l'affichage des nouvelles couleurs et styles.
6. Essayez ensuite de modifier les polices (**android:fontFamily**), les tailles de texte (**android:textSize**), ou encore les couleurs pour personnaliser davantage l'interface.

Partie 10 : Extension - Ajouter une deuxième activité

Cette partie permet de réutiliser la notion d'Intent dans un scénario simple avec une deuxième activité.

1. Créer une nouvelle activité :
 - Menu : **File** → **New** → **Activity** → **Empty Activity**.
 - Nommer l'activité : **WelcomeActivity**, puis valider.

Android Studio génère un fichier **WelcomeActivity.kt** et un layout **activity_welcome.xml**.

2. Modifier le layout :
 - Ouvrir **activity_welcome.xml** en Design.
 - Ajouter un **TextView** avec le texte "**Bienvenue !**" centré à l'écran.
3. Modifier **MainActivity** pour lancer **WelcomeActivity**
 - Dans **btnLogin.setOnClickListener**, créez un Intent vers **WelcomeActivity** et lancez-le avec **startActivity()**.

```
val intent = Intent(this, WelcomeActivity::class.java)
```



```
startActivity(intent)
```

4. Lancer l'application et vérifier le résultat.

BON TRAVAIL 😊